

COMPUTER PROGRAMMING

LAB NO: 08

LAB TITLE

Two-Dimensional Arrays in C++

OBJECTIVE

The objective of this lab is to understand the concept of two-dimensional arrays in C++. This lab helps students learn how to declare and initialize two-dimensional arrays, access array elements using rows and columns, use nested loops with arrays, calculate row and column sums, and find the maximum value from a matrix. Two-dimensional arrays are useful for storing data in tabular form such as rows and columns.

THEORY

A two-dimensional array is a collection of one-dimensional arrays. It is used to store data in the form of rows and columns like a table or matrix. Every element in a two-dimensional array is identified by two indexes: row index and column index.

In C++, indexing starts from 0.

For example:

```
int matrix[3][3];
```

Array Declaration

Syntax

```
dataType arrayName[rows][columns];
```

Array Initialization

Syntax:

```
int matrix[2][2] = {{1,2},{3,4}};
```

Accessing Array Elements

Syntax

```
arrayName[row][column];
```

Nested Loop In 2-D

Nested loops are used with two-dimensional arrays because one loop handles rows while the second loop handles columns.

SOFTWARE

DEV-C++ IDE

C++ Compiler

LAB TASKS

Two-Dimensional Array Declaration and Initialization • Declare a 2D integer array called "matrix" with dimensions 3x3. • Initialize the array with values 1, 2, 3, 4, 5, 6, 7, 8, and 9. • Display the values of the array elements using nested loops.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int matrix[3][3] = {
```

```
{1,2,3},
```

```
{4,5,6},
```

```
{7,8,9}
```

```
}; // matrix ko initialize karna
```

```
for(int i=0; i<3; i++)
```

```
{
```

```
for(int j=0; j<3; j++)
```

```

{
cout << matrix[i][j] << " "; // matrix ke elements print karna
}

cout << endl;

}

return 0;

}

```

Sum of Rows and Columns in a Two-Dimensional Array • Declare and initialize a 3x3 integer array with random values. • Calculate and display the sum of each row and each column separately. • Display the total sum of all the elements in the array.

```

#include <iostream>

using namespace std;

int main()

{

int matrix[3][3] = {

{1,2,3},

{4,5,6},

{7,8,9}

};

int total = 0;

// Row sums calculate karna

for(int i=0; i<3; i++) {

int rowSum = 0;

for(int j=0; j<3; j++) {

```

```

rowSum = rowSum + matrix[i][j];

}

cout << "Row " << i+1 << " Sum = " << rowSum << endl;

}

cout << endl;

// Column sums calculate karna

for(int j=0; j<3; j++)

{

int colSum = 0;

for(int i=0; i<3; i++)

{

colSum = colSum + matrix[i][j];

}

cout << "Column " << j+1 << " Sum = " << colSum << endl;

}

// Total sum calculate karna

for(int i=0; i<3; i++)

{

for(int j=0; j<3; j++)

{

total = total + matrix[i][j];

}

}

}

```

```
cout << endl;

cout << "Total Sum = " << total;

return 0;

}
```

Finding the Maximum Value in a Two-Dimensional Array • Declare and initialize a 4x4 integer array with random values • Find and display the maximum value in the array.

```
#include <iostream>

using namespace std;

int main()

{

int matrix[4][4] = {

{12,7,9,15},

{3,25,11,8},

{14,6,30,10},

{2,18,5,20}

};

int max = matrix[0][0]; // pehli value ko maximum manna

for(int i=0; i<4; i++)

{

for(int j=0; j<4; j++)

{

if(matrix[i][j] > max)

{
```

```
max = matrix[i][j]; // maximum value update karna
}
}

cout << "Maximum Value = " << max;

return 0;

}
```

CONCLUSION

In this lab, we learned the concepts of two-dimensional arrays in C++. We studied declaration, initialization, accessing elements, nested loops, row and column sums, and finding maximum values from arrays. This lab improved our understanding of matrices and two-dimensional data handling in programming.